Slide 1: Problem Statement & Context

Dependency Hell in the AI Era

Target User: Software developers and code reviewers.

The Challenge: Wasting hours configuring local environments (Python virtual environments, Node versions) just to test a single GitHub repository.

Existing Gap: Cloud VMs are too heavy and expensive. Docker requires manual, tedious Dockerfile configuration.

Why Now: The explosion of open-source AI projects means developers are testing more complex, unknown codebases than ever before. We need a zero-friction way to run them safely.

Slide 2: Solution Overview

StackStore: AI-Generated Local Environments

- **What it is:** An OS-level virtualization platform that automatically generates its own infrastructure blueprints.
- **Primary Value:** Zero-configuration, instant local sandboxes for any repository.
- **The Workflow:**

Input URL → AI Analyzes Code → C++ Engine Builds Sandbox

- **The Distinct Edge:** It combines the intelligence of Large Language Models to infer dependencies with the raw performance of native Linux system calls, resulting in zero cloud compute costs.

Slide 3: Key Features & User Flow

Key Features & User Flow

Feature 1: AI Dependency Inference: Parses repositories without needing a pre-existing Dockerfile.

Feature 2: Smart Secrets Management: Automatically detects missing .env variables and prompts the user before execution.

Feature 3: Kernel-Level Isolation: Executes code in a secure, chroot-style jail completely severed from the host OS.

The Flow:

1. Desktop Client (UI)
User pastes a GitHub URL into the desktop client.

2. AI Orchestrator
resolves dependencies and generates a JSON blueprint.

3. C++ Engine
boots the isolated environment natively.

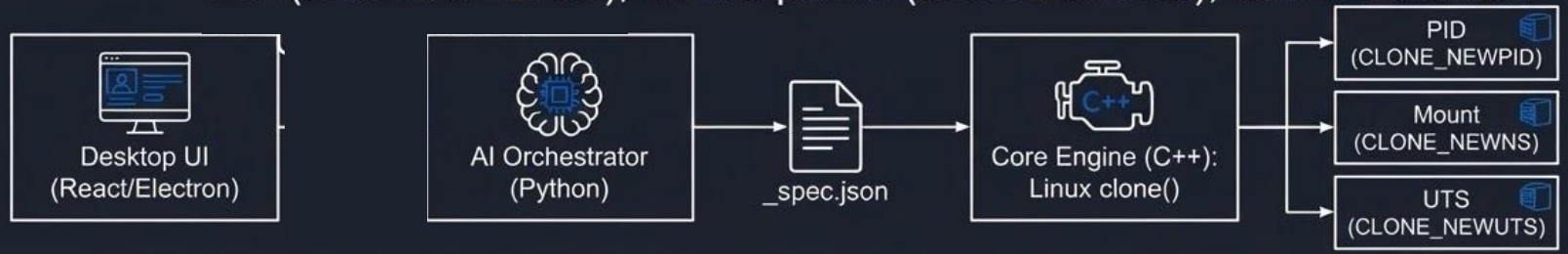
Slide 4: Architecture & Technology Approach

Decoupled Microservice Ecosystem

- **Desktop UI (React/Electron):** A sleek, cross-platform control panel for user input and secrets management.

- **AI Orchestrator (Python):** The intelligence layer that reads the repository architecture and generates a standardized `_spec.json` blueprint.

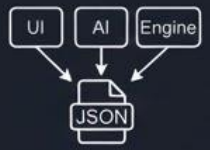
- **Core Engine (C++):** A headless, high-performance binary that parses the JSON and executes raw Linux `clone()` syscalls to isolate Process IDs (`CLONE_NEWPID`), Mount points (`CLONE_NEWNS`), and Hostnames.



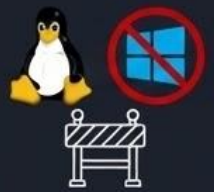
Slide 5: Design Decisions & Trade-offs

Design Decisions & Trade-offs

Prioritized Local Execution: We intentionally dropped cloud-hosted VMs (which are easier to build) in favor of a local C++ kernel engine to ensure strict data privacy and zero-cost scaling.



Decoupled Architecture: We separated the UI, AI, and Engine into distinct microservices interacting only via JSON.

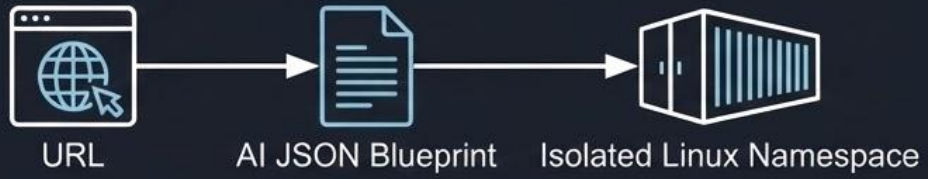


Trade-off & Constraint: Building raw Linux namespaces means the engine cannot currently run natively on Windows, requiring cross-platform handoffs during development.

Slide 6: Current Status, Limitations & Next Steps

Current Status & Roadmap

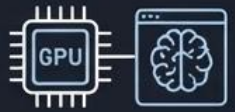
- **Fully Functional:**



The end-to-end pipeline successfully executes code.

- **Known Limitations:** The virtualization engine currently requires a native Linux OS (like Ubuntu) to fire the `clone()` system calls.
- **Next Steps (V2):**

1. Windows Subsystem for Linux (WSL) integration for native Windows support.



2. GPU hardware passthrough for local AI model training inside the sandbox.